

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL
CARRERA DE INGENIERÍA DE SOFTWARE

APLICACIONES WEB [111] – 5TO NIVEL – PARALELO A

TEMA:

Informe Técnico del Examen Unidad IV

**Modelo Vista Controlador y Servicios Web: API REST, Autenticación JWT
y Consumo de APIs Externas**

ESTUDIANTE: PANAMA MURILLO MOISES ANTONIO
DOCENTE: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.
REPOSICION: <https://github.com/MoisesPanama/biblioteca-u4-base>
PERIODO: 2026–2027 PPA

Contents

1	Introducción y Objetivos	2
2	Matriz Tecnológica y Entorno	2
3	Desarrollo Técnico	2
3.1	B1. Endpoints REST y Envoltorio Estándar	2
3.2	B2. Autenticación JWT y Seguridad por Roles	2
3.3	B3. API Externa con Cache-Aside y Manejo de Fallos	2
3.4	B4. Pruebas de Integración	3
4	Comandos de Compilación del Informe	3
5	Conclusiones	3

1 Introducción y Objetivos

El presente informe documenta la solución técnica desarrollada para el examen práctico de la **Unidad IV** en la asignatura Aplicaciones Web. El proyecto base `biblioteca-u4-base` fue adaptado para exponer una API RESTful robusta sobre **Java 25** y **Spring Boot 4.1.1**, aplicando los principios del estilo arquitectónico REST [1].

Los objetivos implementados abarcan:

- Construcción de controladores RESTful con soporte de filtrado dinámico, paginación y envoltorio estandarizado `ApiResponse` [2].
- Seguridad *stateless* mediante tokens JWT (*JSON Web Token*) y control de acceso basado en roles para microservicios [3].
- Integración con la API externa OpenLibrary utilizando el patrón *Cache-Aside* sobre Redis 8 y manejo uniforme de errores.
- Pruebas de integración automatizadas utilizando Testcontainers 2.x con PostgreSQL 18.

2 Matriz Tecnológica y Entorno

Table 1: Componentes de software y versiones utilizadas.

Componente	Versión	Función / Observación
Java	25 (LTS)	Lenguaje base (SDK 25)
Spring Boot	4.1.1	Framework principal (<code>spring-boot-starter-webmvc</code>)
Spring Data JPA	4.1.x	Persistencia y consultas dinámicas mediante Specifications
Flyway	12.4.0	Migración de esquemas SQL (V1, V2, V3)
PostgreSQL	18	Base de datos relacional en Docker
Redis	8	Memoria caché (<i>Cache-Aside</i>)
Testcontainers	2.0.5	Entorno aislado para pruebas de integración

3 Desarrollo Técnico

3.1 B1. Endpoints REST y Envoltorio Estándar

Se implementó el controlador `LibroController` para gestionar las peticiones sobre los libros. La consulta `GET /api/v1/libros` acepta parámetros de filtrado (`titulo`, `categoriaId`, `anioDesde`) delegados a `LibroService.buscar(...)`. Los datos devueltos se envuelven en el objeto `ApiResponse` junto con la metadata de paginación `PageMeta`, siguiendo las recomendaciones de diseño en servicios RESTful [2].

3.2 B2. Autenticación JWT y Seguridad por Roles

La autenticación se implementó de forma *stateless*. La emisión de tokens se gestiona en `AuthController`, validando credenciales y retornando un JWT firmado con algoritmo HMAC-SHA [3]. El filtro `JwtAuthenticationFilter` intercepta cada solicitud para establecer el contexto de seguridad. La protección de endpoints se configuró con `@PreAuthorize("hasRole('ADMIN')")` para la creación de recursos (`POST /api/v1/libros`).

3.3 B3. API Externa con Cache-Aside y Manejo de Fallos

Se desarrolló el cliente HTTP `OpenLibraryClient` para consultar metadatos adicionales por ISBN.

- **Caché:** Se configuró el espacio de caché `openlibrary` con un TTL de 24 horas en Redis.
- **Resiliencia:** Ante un código 404 del proveedor externo, los campos externos retornan `null` manteniendo la respuesta 200 OK del libro local. Fallos de red o errores 5xx lanzan una `ServicioExternoException`, mapeada automáticamente a un estado 502 Bad Gateway.

3.4 B4. Pruebas de Integración

En `LibroControllerIT` se implementaron pruebas automatizadas sobre la base de datos PostgreSQL en contenedor:

1. Verificación del estado 200 OK y estructura del envoltorio `ApiResponse` en el listado.
2. Comprobación del formato de error (`ProblemDetails`) al solicitar un recurso inexistente (404 Not Found).
3. Validación de respuesta 400 Bad Request ante peticiones con cuerpo inválido.

4 Comandos de Compilación del Informe

Para dar cumplimiento al criterio de evaluación obligatorio, la cadena de compilación de este documento desde la terminal en el directorio `docs/` es la siguiente:

```
cd docs
pdflatex informe.tex
bibtex informe
pdflatex informe.tex
pdflatex informe.tex
```

5 Conclusiones

La realización de esta práctica permitió afianzar la arquitectura en capas con Spring Boot 4.1.1 y Java 25. Se verificó la importancia de utilizar convenciones REST puras [1], separación de responsabilidades en la autenticación JWT y desacoplamiento de servicios externos mediante el patrón caché [2, 3].

References

- [1] R. T. Fielding and R. N. Taylor, “Principled design of the modern web architecture,” *ACM Transactions on Internet Technology (TOIT)*, vol. 2, no. 2, pp. 115–150, 2002. DOI: <https://doi.org/10.1145/514183.514185>.
- [2] C. Pautasso, O. Zimmermann, and F. Leymann, “Restful web services vs. "big" web services: Making the right architectural decision,” in *Proceedings of the 17th International Conference on World Wide Web (WWW '08)*, pp. 805–814, ACM, 2008. DOI: <https://doi.org/10.1145/1367497.1367606>.
- [3] D. Taibi, V. Lenarduzzi, and C. Pahl, “Architectural patterns and security strategies in web microservices,” *IEEE Access*, vol. 8, pp. 164200–164215, 2020. DOI: <https://doi.org/10.1109/ACCESS.2020.3017770>.